SALAMATYAR

مستند پروژهٔ سلامت‌یار

دستیار یادآوری سلامت روی تلگرام — معماری، NLP مدل‌محور، زمان‌بندی دقیق و عملیات

Cloudflare Workers + D1 · نسخهٔ ۱٫۰ · ۲۱۶/۲۱۶ آزمون موفق · ۱۱ آبان ۱۴۰۵

۱. معرفی و خلاصهٔ پروژه

سلامت‌یار (SalamatYar) یک ربات تلگرامی فارسی‌زبان و راست‌به‌چپ برای یادآوری‌های شخصی سلامت است: دارو، نوبت و آزمایش، نوشیدن آب و برنامهٔ روزانه. ربات روی Cloudflare Workers اجرا می‌شود، داده‌ها را در D1 (SQLite مدیریت‌شده) نگه می‌دارد و هیچ سرور یا پایگاه‌دادهٔ خارجی ندارد؛ کل سرویس در یک Worker بدون حالت و یک دیتابیس خلاصه می‌شود. هویت کاربر همان شناسهٔ عددی تلگرام است، همهٔ متن‌ها فارسی است و تقویم و اعداد جلالی ارائه می‌شوند.

این سند، نسخهٔ فشردهٔ مستند کامل پروژه است و در حداکثر شش صفحه تصویر دقیق و به‌روزِ سیستم را می‌دهد: معماری و ماژول‌ها، چرخهٔ پیام و لایهٔ NLP مدل‌محور، موتور زمان‌بندی، مدل داده، امنیت، وضعیت آزمون‌ها و رویهٔ استقرار. شرح گسترده‌تر هر بخش (جداول ستون‌به‌ستون و متن کامل نمونه‌ها) در نسخهٔ بلند سند موجود است.

جدول ۱: شناسنامهٔ پروژه

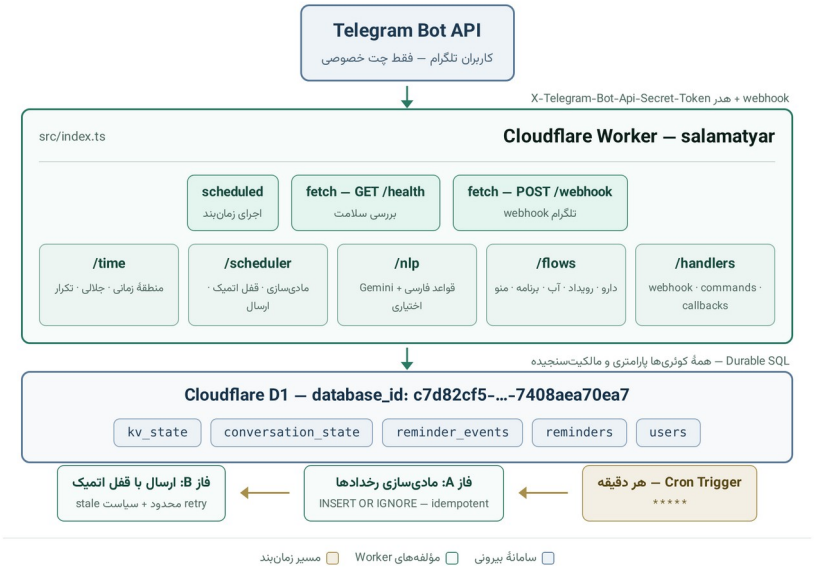
ویژگی	مقدار
نام و نسخه	سلامت‌یار — SalamatYar، نسخهٔ ۱٫۰٫۰
پلتفرم	Cloudflare Workers + D1 · Telegram Bot API (Webhook + Cron Trigger)
زبان و جهت	فارسی (fa-IR) و RTL · تقویم جلالی · منطقهٔ پیش‌فرض `Asia/Tehran`
NLP	قاعده‌محور آفلاین + Gemini اختیاری (`LLM_PROVIDER=auto` · مدل `gemini-3.5-flash-lite`)
کیفیت	۲۱۶/۲۱۶ آزمون موفق (۱۰۳ واحد + ۱۱۳ یکپارچگی) · tsc و eslint بدون خطا
استقرار	کد کامل و آماده؛ deploy فقط با اجازهٔ مالک و اعتبارنامهٔ `CLOUDFLARE_API_TOKEN`

قابلیت‌های اصلی در یک نگاه:

- 💊 دارو: چند ساعت در روز، تکرار روزانه/هفتگی/چندروزی/ماهانه، توقف و ادامه، ویرایش، حذف دومرحله‌ای و تعویق
- 🩺 نوبت و 🧪 آزمایش: رویداد یک‌باره با تاریخ و ساعت، مکان اختیاری و پیش‌یادآوری چندگانه (مثلاً یک روز و یک ساعت قبل)
- 💧 آب: پنجرهٔ زمانی دلخواه با فاصلهٔ تکرار ۵ دقیقه تا ۳ ساعت؛ روشن/خاموش کردن با یک دکمه
- 📅 برنامهٔ روز و هفته: امروز، فردا، روز مشخص و نمای ۷ روزه با وضعیت هر رخداد
- 🗣️ جملهٔ آزاد فارسی: تحلیل ساخت‌یافته کارت تأیید ثبت فقط با تأیید کاربر
- 💬 مکالمهٔ چندمرحله‌ای: تکمیل اطلاعات ناقص، ارجاع حسی («همون»)، چند درخواست در یک پیام، انقضای خودکار ۳۰ دقیقه

۲. معماری سیستم

نقطهٔ ورود index.ts درخواست‌ها را میان /webhook و /health تقسیم می‌کند و Cron Trigger (هر دقیقه) را با waitUntil اجرا می‌کند. کد به لایه‌های مسئولیت‌مند تقسیم شده است: /telegram (فراخوانی Bot API با retry، سازندهٔ کیبوردها، اعتبارسنجی callback_data و همهٔ متن‌های فارسی در texts.ts)، /handlers (اعتبارسنجی رمز webhook با مقایسهٔ ثابت‌زمان و مسیریابی دستورها و callbackها)، /flows (جریان‌های دارو، نوبت، آب، برنامه و منوها)، /nlp (لایهٔ انتزاعی Provider و پیاده‌سازی‌ها)، /scheduler و /db و /services و /conversation و /time. هیچ کامپوننتی جز لایهٔ /db با SQL سروکار ندارد.



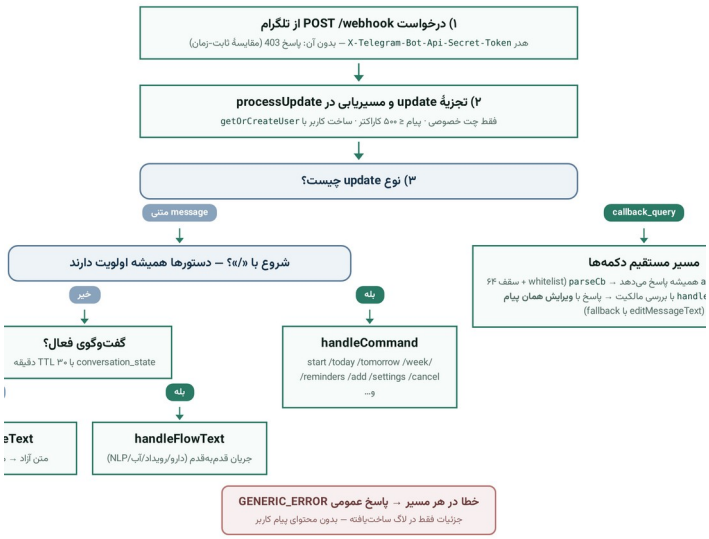
شکل ۱: معماری کلی: تلگرام → Worker (لایه‌های پردازش) → D1 و Cron دو فاز

۳. چرخهٔ پیام و لایهٔ NLP مدل‌محور

هر پیام پس از بررسی رمز هدر X-Telegram-Bot-Api-Secret-Token به یکی از سه مسیر می‌رود: دستور شناخته‌شده مستقیماً اجرا می‌شود؛ callback دکمه‌ها پس از parseCb (الگوی چهاربخشی، سفهرس آسان‌نشانی، سقف ۶۴ بایت) و بررسی مالکیت، پیام اصلی را با editMessageText ویرایش می‌کند؛ و متن آزاد فارسی به لایهٔ NLP می‌رود. اگر گفت‌وگوی فعالی وجود داشته باشد (D1 با ۳۰ TTL دقیقه)، پیام به همان جریان هدایت می‌شود.

جدول ۲: دستورهای ربات

دستورها	عملکرد
/start، /help، /menu	شروع و خوش‌آمد، راهنمای کامل و نمایش مجدد منوی اصلی
/today · /plan · /tomorrow · /week	برنامهٔ امروز، فردا و نمای فشردهٔ ۷ روز آینده با وضعیت هر رخداد
/reminders · /list	فهرست همهٔ یادآوری‌ها با دکمه‌های مدیریت هر مورد
/add، /meds، /water، /appts	منوی یادآوری جدید و میان‌بر جریان دارو، آب و نوبت/آزمایش
/settings	منطقهٔ زمانی، حریم خصوصی و حذف کامل داده‌ها با تأیید دومرحله‌ای
/cancel	لغو گفت‌وگوی جاری؛ دستور ناشناس هم پاسخ «دستور را نمی‌شناسم» به‌همراه راهنما می‌گیرد



شکل ۲: چرخهٔ پیام: سه مسیر دستور، callback و متن آزاد (با اولویت گفت‌وگوی فعال)

معماری NLP از نوع LLM-first است: در حالت‌های gemini و auto، جملهٔ کاربر نخست به مدل زبانی می‌رود؛ خروجی با NLU_RESPONSE_SCHEMA (خروجی ساخت‌یافته و temperature=0) گرفته و با normalizeNlu سخت‌گیرانه اعتبارسنجی می‌شود. تاریخ‌های نسبی («فردا»، «هفتهٔ بعد») در بک‌اند و به‌صورت قطعی به UTC تبدیل می‌شوند و مدل هیچ دسترسی‌ای به SQL و D1 ندارد. عملیات حساس فقط با کارت تأیید انجام می‌شود و در هر خطا (timeout، خروجی نامعتبر، اتمام سهمیه) مسیر امن به Rule Parser آفلاین بازمی‌گردد. سهمیهٔ مدل ۶۰ فراخوان روزانه برای هر کاربر است (LLM_DAILY_LIMIT) و مدل پیش‌فرض gemini-3.5-flash-lite دست‌نخورده مانده است.

جدول ۳: حالت‌های LLM_PROVIDER

حالت	رفتار
auto	اول Gemini (اگر کلید باشد) و در خطا بازگشت به قواعد — پیش‌فرض
gemini	همیشه مدل اول؛ قواعد فقط به‌عنوان مسیر امن خطا
workersai	مدل Workers AI با بایندینگ AI اختیاری (@cf/meta/llama-3.3-70b-instruct-fp8-fast)
rules	فقط Parser قاعده‌محور آفلاین — بدون هیچ فراخوانی شبکه‌ای

۴. موتور زمان‌بندی و پایگاه داده

ارسال سر وقت با Cron هر دقیقه در دو فاز انجام می‌شود: فاز A (materializeDue) رخدادهای سررسید را با INSERT OR IGNORE و کلید یکتای محلی occurrence_key به‌صورت idempotent می‌سازد و فاز B (claimEventForDelivery) هر رخداد را با UPDATE شرطی (فقط وقتی delivery_status خالی است) اتمیک قفل و ارسال می‌کند؛ در نتیجه اجرای تکراری cron یا هم‌زمانی، ارسال دوباره نمی‌سازد. خطای گذرا تا ۳ بار تلاش می‌شود (MAX_SEND_ATTEMPTS)، حداکثر ۲۵ ارسال در هر تیک (MAX_BATCH) و رخدادهای بیش از ۶ ساعت عقب‌افتاده طبق سیاست STALE_HOURS ارسال نمی‌شوند. تعویق «۱۰ دقیقه دیگر» رخداد جدیدی با کلید #s<epoch> می‌سازد؛ کلیدهای رخداد روزانه #main و پیش‌یادآوری‌ها a1440 هستند.

پایگاه دادهٔ D1 (شناسهٔ واقعی در wrangler.jsonc) پنج جدول دارد و همهٔ کوئری‌ها صددرصد پارامتری است؛ هر خواندن و نوشتن با user_id سنجیده می‌شود. قاعدهٔ تکرار در ستون recurrence به‌صورت JSON ذخیره می‌شود: daily، weekly

با آرایهٔ روزها (۰=شنبه)، interval بر حسب دقیقه با پنجرهٔ زمانی، ndays (۲ تا ۱۸۰ روز)، monthly و NULL برای رویداد یک‌باره.

جدول ۴: جدول‌های D1 و نقش هر یک

جدول	نقش
users	هویت بر پایهٔ telegram_user_id یکتا، chat_id، منطقهٔ زمانی و زبان
reminders	یادآوری‌ها: timezone، local_time، recurrence (JSON)، title، category، بازهٔ شروع/پایان، advance_minutes، status
reminder_events	رخدادهای مادی‌شده با occurrence_key یکتا و وضعیت تحویل (قفل اتمیک ارسال)
conversation_state	وضعیت گفت‌وگوی چندمرحله‌ای با TTL و انقضای ۳۰ دقیقه
kv_state	شمارندهٔ سهمیهٔ روزانهٔ LLM (کلیدهای llmq:*) و وضعیت‌های عمومی

جدول ۵: متغیرهای کلیدی پیکربندی و مقادیر فعلی

متغیر	پیش‌فرض	اثر
DEFAULT_TIMEZONE	Asia/Tehran	منطقهٔ زمانی پیش‌فرض کاربران جدید
LLM_PROVIDER	auto	سیاست NLP: auto · gemini · workersai · rules
GEMINI_MODEL	gemini-3.5-flash-lite	مدل پیش‌فرض Gemini (دست‌نخورده)
LLM_TIMEOUT_MS · LLM_DAILY_LIMIT	60 · 15000	سقف انتظار پاسخ و سقف فراخوانی روزانهٔ هر کاربر (۰=بی‌سقف)
STALE_HOURS · MAX_SEND_ATTEMPTS	3 · 6	سیاست رخدادهای عقب‌افتاده و سقف تلاش ارسال هر رخداد
MAX_BATCH · SNOOZE_MINUTES	10 · 25	سقف ارسال در هر تیک cron و فاصلهٔ پیش‌فرض تعویق

۵. امنیت، زمان‌وشماری و تضمین کیفیت

- رمز webhook با مقایسهٔ ثابت‌زمان بررسی می‌شود؛ رمزها فقط در Secrets (و .dev.vars محلی) هستند و هرگز در کد یا لاگ نمی‌روند.
- همهٔ کوئری‌ها پارامتری‌اند و هر عملیات با مالکیت user_id سنجیده می‌شود؛ کاربر فقط به ردیف‌های خودش دسترسی دارد.
- حذف کامل داده‌ها از داخل ربات (/settings) با تأیید دومرحله‌ای ممکن است؛ فقط حداقل دادهٔ سلامت ذخیره می‌شود.
- هیچ توصیهٔ پزشکی‌ای داده نمی‌شود؛ سؤال‌های دوز/تشخیص/درمان با پاسخ مرزی هدایت و ثبت نمی‌شوند.

زمان در هر کاربر با منطقهٔ IANA خودش (پیش‌فرض Asia/Tehran) محاسبه می‌شود؛ تقویم جلالی و ارقام فارسی در نمایش و پارس ورودی پشتیبانی می‌شوند و محاسبهٔ تکرار روی تاریخ محلی انجام می‌گردد تا تغییر منطقه یا DST گذشته را خراب نکند. ساعت منطقه در ستونِ timezone خودِ یادآوری نیز ذخیره می‌شود.

آزمون‌ها با vitest و @cloudflare/vitest-pool-workers روی workerd واقعی اجرا می‌شوند؛ D1 همان SQLite محلی miniflare است، زمان قطعی است (FIXED_NOW) و تلگرام و Gemini با ماک fetch ایزوله‌اند — هیچ درخواست واقعی از آزمون‌ها بیرون نمی‌رود. در اجرای کامل ۱۱ آبان ۱۴۰۵ هر ۱۸ فایل آزمون موفق بود (۲۱۶ آزمون در حدود ۲۲ ثانیه).

جدول ۶: پوشش آزمون به تفکیک حوزه (جمع: ۲۱۶)

حوزه	تعداد	پوشش اصلی
NLP واحد (nlp، intervals، dates)	۶۱	پارس ساعت و فاصله‌ها، تاریخ نسبی در مسیریابی، مرز پزشکی
قواعد تکرار و ساعت (recurrence، clock)	۳۰	پنج قالب تکرار، DST و پرش نیمه‌شب، کلید رخداد
فارسی و جلالی (persian)	۱۲	ارقام، جلالی↔میلادی با رفت‌وبرگشت و مقایسه با ICU
دستورها و رگرسیون (commands-plan، onboarding)	۲۴	رگرسیون `/today`، دیتابیس خالی/پُر، idempotency، دو کاربر
امنیت و LLM-first (nlp-security، llm-first، gemini-provider)	۳۶	کارت تأیید، fallback، SQL injection، 403، امن، retry و سهمیه
زمان‌بندی و جریان‌ها (scheduler و ۶ فایل دیگر)	۵۳	قفل اتمیک، cron تکراری، ده سناریوی الزامی، آب و نوبت

۶. استقرار، رویداد رفع اشکال و نقشهٔ راه

استقرار به‌صورت خودکار انجام نمی‌شود و طبق قاعدهٔ همکاری فقط با اجازهٔ صریح مالک انجام می‌شود؛ تنها پیش‌نیاز باقی‌مانده، اعتبارنامهٔ مالک با دسترسی Workers Scripts: Edit و D1: Edit است. رمزهای موجود (`TELEGRAM_BOT_TOKEN` و `TELEGRAM_WEBHOOK_SECRET`) هنگام deploy به‌طور خودکار حفظ می‌شوند و `GEMINI_API_KEY` اختیاری است. رویهٔ کامل:

جدول ۷: رویهٔ استقرار در تولید

گام	دستور / اقدام
۱	`npx wrangler d1 migrations apply salamatyar --remote` — اعمال مهاجرت‌های تولید
۲	`npx wrangler deploy` — استقرار Worker (شناسهٔ D1 از قبل در `wrangler.jsonc` ثبت است)
۳	setWebhook با همان `TELEGRAM_WEBHOOK_SECRET` و ثبت منوی دستورها با setMyCommands
۴	بررسی `GET /health` (باید «salamatyar is up» برگرداند) و getWebhookInfo بدون `last_error_message`
۵	آزمون زندهٔ `/today` و اعلان واقعی با رصد `npx wrangler tail`

رویداد مهم اخیر: دستور `/today` با خطای «undefined (reading `occurrence_key`)» شکست می‌خورد؛ ریشه‌یابی نشان داد تبدیل نوع مستقیم ردیف‌های تخت D1 به ساختار تودرتوی DayEvent بدون نگاشت واقعی علت است. اصلاح ریشه‌ای با نگاشت دستی ستون‌ها انجام شد، ۱۴ آزمون رگرسیون افزوده شد و دیتابیس بازسازی نشد — هیچ داده‌ای حذف نشد (دو کاربر، هفت یادآوری و دو رخداد حفظ شدند) و مهاجرت جدیدی لازم نبود.

نقشهٔ راه دورهٔ بعد:

- استقرار تولید و تأیید زنده — بلوکه روی `CLOUDFLARE_API_TOKEN` مالک؛ بلافاصله پس از آن ثبت منوی دستورها
- پایش عملیاتی با `wrangler tail` و observability؛ رصد رویدادهای `llm_usage` و `llm_quota_exceeded`
- بهبود تدریجی NLP بر اساس نمونه‌های واقعی کاربران و گسترش Rule Parser

> *این سند نسخهٔ فشرده است؛ شرح کامل هر بخش — جداول ستون‌به‌ستون، نمونه‌ها و متن کامل رویداد رفع اشکال — در «مستند جامع» (۳۰ صفحه) همان پروژه آمده است.*